

Table of contents

[Documentation](#) [Target](#) [Target Guide](#)

Profile attributes

Last update: February 23, 2024



Bookmark



Copy link

TOPICS: Audiences

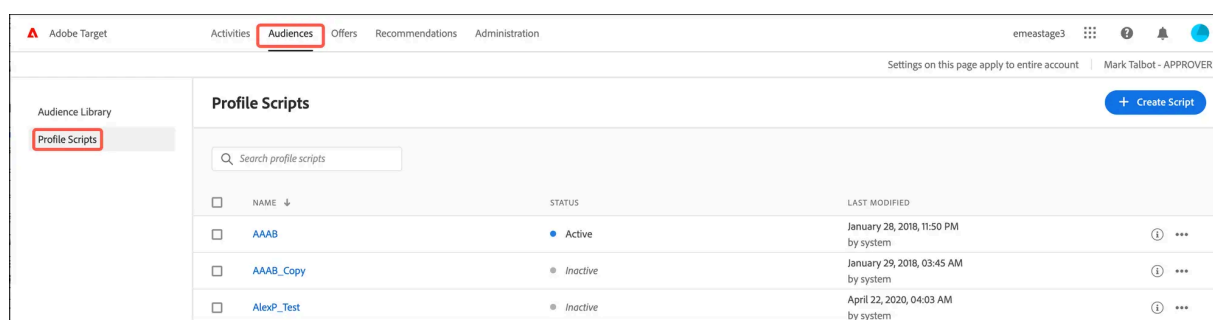
Profile attributes in Adobe Target are parameters that are specific to a visitor. These attributes are stored in the visitor's profile to provide information about the visitor that can be used in your activities.

A user profile contains demographic and behavioral information of a web page visitor. This information can include age, gender, products purchased, last time of visit, and so on. Target uses this information to personalize the content it serves to that visitor.

As a visitor browses your website, or when the visitor returns for another session, the saved profile attributes in the profile can be used to target content or log information for segment filtering.

To set up profile attributes:

1. Click **Audiences > Profile Scripts**.

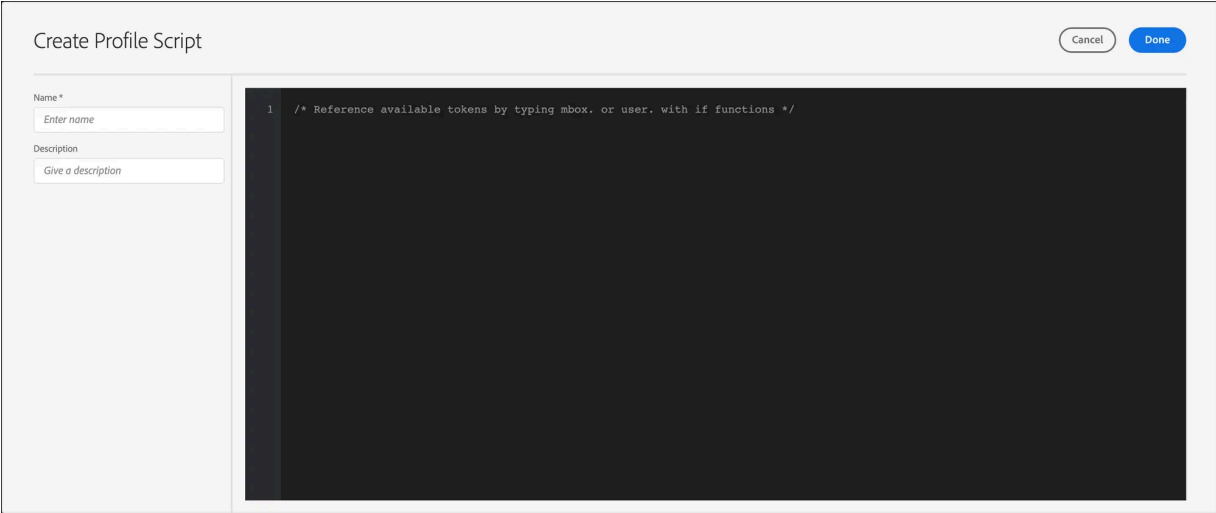


2. Click **Create Script**.

Was this content helpful?

Detailed feedback
options





The following types of profile attributes are available:

Parameter Type	Description
mbox	Passed in directly through page code when creating the mbox. See Pass Parameters to a Global Mbox in the <i>Target Developer Guide</i>... Note: Target has a limit of 50 unique profile attributes per mbox call. If you must pass more than 50 profile attributes to Target, pass them using the Profile Update API method. For more information, see Update profiles in the <i>Target Developer Guide</i>.
Profile	Defined directly with a JavaScript code snippet. These snippets can store running totals, such as total money spent by consumer, and are executed on each mbox request. See <i>Profile script attributes</i> below.

Profile script attributes

Define a profile script attribute with its associated JavaScript code snippet.

You can use profile scripts to capture visitor attributes across multiple visits. Profile scripts are code snippets defined within Target using a form of server-side JavaScript. For example, you might use a profile script to capture how frequently a visitor visits your site, and when that visitor last visited.

Prof visit

Was this content helpful?

Detailed feedback options

×

Create profile scripts

Profile scripts are available under the Audiences tab in the Target interface.

To add a profile script, click the **Profile Scripts** tab, **Create Script**, then write your script.

Or

To copy an existing profile script, from the Profile Scripts list, click the ellipsis icon for the desired script, then click **Duplicate**.

You can then edit the audience to create a similar audience.

Profile scripts run profile attribute “catchers” on each location request. When a location request is received, Target determines which activity should run and displays content that is appropriate to that activity and that experience. Target also tracks the success of the activity and runs any relevant profile scripts. This process lets you track information about the visit, such as the visitor’s location, time of day, number of times that visitor has been to the site, if they’ve purchased before, and so on. This information is then added to the visitor’s profile so you can better track that visitor’s activity on your site.

Profile script attributes have the `user .` tag inserted before the attribute name. For example:

```
if (mbox.name == 'Track_Interest') {
  if (profile.get('model') == "A5" && profile.get('subcat') == "KS6") {
    return (user.get('A5KS6') || 0) + 1;
  }
}
```

Keep the following information in mind:

- Refer to profile script attributes (including itself) in the code with `user.get('parameterName')`.
- Save variables that can be accessed the next time the script is run (on the next mbox request) with `user.setLocal('variable_name', 'value')`. Reference the variable with `user.getLocal('variable_name')`. This process is useful for situations where you want to reference the date and time of the last request.

These values persist just like a profile script, but you only have access to them within the script they were set.

- Parameters and values are case-sensitive. Match the case of the parameters and values you receive during the activity or test.
- See the “JavaScript reference for script profile parameters” section below for more JavaScript syntax.
- The parameter remains in the profile after disabling the script. Users whose profiles already contain a

Was this content helpful?

•

Detailed feedback
options

×

- Creating dependent profile scripts that use the result of one profile script in another profile script is not recommended. The order of profile script execution is not guaranteed.

Viewing profile script information cards

You can view profile script information pop-up cards similar to offer information cards. These profile script information cards let you view the list of activities that reference the selected profile script, along with other useful metadata.

For example, the following profile script information card is accessed by clicking the Info icon for the desired profile script from the list (Audiences > Profile Scripts).

The Script Info tab contains the following information: Name, Description, and script code.

The screenshot shows the Adobe Target interface with the 'Audiences' tab selected. On the left, the 'Profile Scripts' list is visible. On the right, a detailed view of the 'Harpreet_17Aug_AutoDisable' script is shown, highlighted with a red box. The detailed view includes the script ID (ID#1787005), name, description, and the script code.

Click **View full details** to see the audiences and activities that reference the selected profile script.

The screenshot shows the 'Harpreet_17Aug_AutoDisable' script details page. The page includes a 'Script' tab with the script code, a 'Details' tab with metadata, and a 'Usage' section highlighted with a red box. The 'Usage' section shows the script name and the number of activities (2) that reference it.

Was this content helpful?

Detailed feedback
options

NOTE

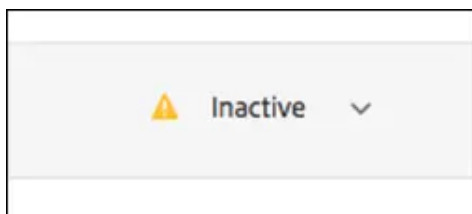
The Script Usage tab does not display activities that reference the selected profile script in the following situations:

- The activity is in the Draft state.
- The content or offer used in the activity uses script variables (either an inline offer within the activity or an offer within the Offer library).

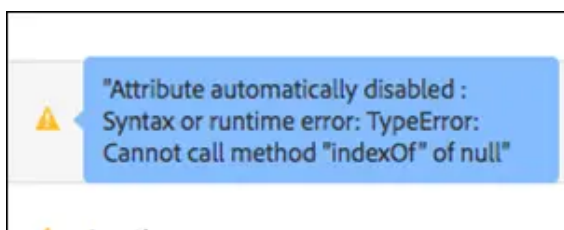
Target disables profile scripts in certain situations

Target automatically disables profile scripts in certain situations, such as if they take too long to execute or have too many instructions.

When a profile script is disabled, a yellow alert icon displays next to the profile script in the Target UI, as illustrated below:



On hover, details on the error display, as illustrated below:



Typical reasons for the system to disable profile scripts include the following:

- An undefined variable to referenced.
- An invalid value is referenced. This error is often caused by referencing URL values and other user-inputted data without proper validation.
- Too many JavaScript instructions are used. Target has limit of 2,000 JavaScript instructions per script, but this limit cannot simply be calculated by manually reading the JavaScript. For example, Rhino treats all function calls and "new" calls as 100 instructions. Any call to any function consumes 100 instructions. Also, the size of any entry data, such as URL values, can impact the instructions count.

Was this content helpful?

Detailed feedback
options

Best practices

The following guidelines are meant to help write simplified profile scripts that are as error-failing-free as possible by writing code that fails gracefully so the scripts are processed without forcing a system-script-halt. These guidelines are a result of best practices that have been proven to run efficiently. These guidelines are to be applied alongside principles and recommendations drawn by the Rhino development community.

- Set current script value to a local variable in the user script, set a failover to a blank string.
- Validate the local variable by ensuring it is not a blank string.
- Use string-based manipulation functions vs. Regular Expressions.
- Use limited for loops vs. open ended for or while loops.
- Do not exceed 1,300 characters or 50 loop iterations.
- Do not exceed 2,000 JavaScript instructions. Target has limit of 2,000 JavaScript instructions per script, but this limit cannot simply be calculated by manually reading the JavaScript. For example, Rhino treats all function calls and "new" calls as 100 instructions. Also, the size of any entry data, such as URL values, can impact the instructions count.
- Be mindful of not only the script performance, but the combined performance of all scripts. As best practice, Adobe recommends fewer than 5,000 instructions in total. Counting the number of instructions is not obvious, but the important thing to note is that scripts exceeding 2,000 instructions are automatically disabled. The number of active profile scripts should not exceed 300. Each script is executed with every mbox call. Run only as many scripts as needed.
- In a regex, having dot-star in the beginning (for example: `/. *match/`, `/a | . *b/`) is almost never needed. The regex search starts from all positions in a string (unless bound with `^`), so dot-star is already assumed. The script execution can be interrupted if such a regex is matched to a long enough input data (which can be as low as several hundred characters).
- If all fails, wrap script in a try/catch.
- The following recommendations can help you limit profile script complexity. Profile scripts can execute a limited number of instructions.

As best practice:

- Keep profile scripts small and as simple as possible.
- Avoid regular expressions or use only simple regular expressions. Even simple expressions can take many instructions to evaluate.

Was this content helpful?

Detailed feedback
options



- Profile scripts should be performance-tested before being added to Target. All profile scripts execute on every mbox request. If profile scripts do not execute correctly, mbox requests take longer to execute, which can impact traffic and conversion.
- If profile scripts become too complex, consider using [response tokens](#) instead.
- See the JS Rhino engine documentation for more information.

Debug profile scripts

The following methods can be used to debug profile scripts:

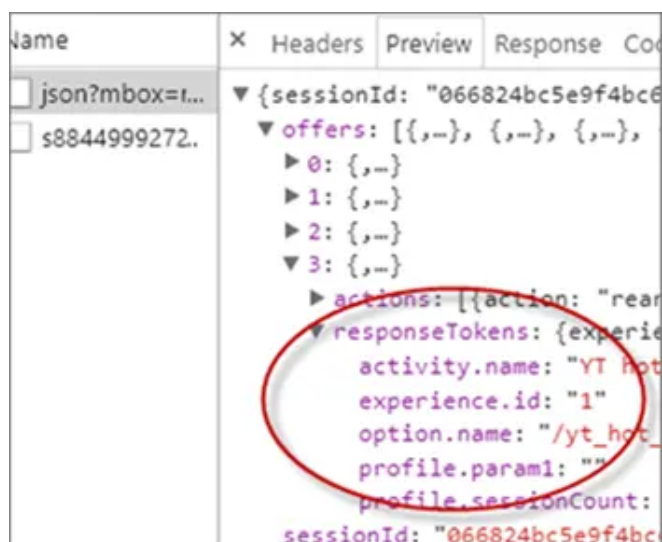
NOTE

Using console.log within a profile script does not output the profile value, because profile scripts execute server-side.

- **Add profile scripts as response tokens to debug profile scripts:**

In Target, click **Administration**, click **Response Tokens**, then enable the profile script you want to debug.

Anytime you load a page for your site with Target on it, part of the response from Target contains your value for the given profile script, as shown below:



- **Use the mboxTrace debugging tool to debug profile scripts.**

This method requires an authorization token that you can generate by clicking **Target** > **Administration** > **Implementation** > **Generate Authorization Token** in the Debugger tools section.

You then add these two parameters to your page URL after the "?":

Was this content helpful?

Detailed feedback
options



Adding these parameters is a little more informative than the response token because you get a before-executed snapshot and an after-snapshot of your profile. It also shows all your available profiles.



Profile script FAQ

Is it possible to use profile scripts to capture information from a page that resides in a data layer?

Profile scripts are unable to read the page directly because they are executed server side. The data must be passed in through an mbox request or through other [methods of getting data into Target](#). After the data is in Target, profile scripts can read the data as an mbox parameter or profile parameter.

JavaScript reference for script profile parameters

Simple Javascript knowledge is required to effectively use script profile parameters. This section serves as a quick reference to make you productive with this functionality in just a few minutes.

Script Profile Parameters are found under the mboxes/profiles tab. You can write Javascript programs that return any Javascript type (string, integer, array, and so forth).

Script profile parameter examples

Name: *user.recency*

Was this content helpful?

Detailed feedback
options

```

    user.setLocal('lastPurchaseTime', new Date().getTime());
  }
  var lastPurchaseTime = user.getLocal('lastPurchaseTime');
  if (lastPurchaseTime) {
    return ((new Date()).getTime() - lastPurchaseTime) / dayInMillis;
  }

```

Creates a variable for day as measured in milliseconds. If the mbox name is orderThankyouPage, set a local (invisible) user profile attribute named lastPurchaseTime to show the value of the current date and time. The value of last purchase time is read, and if defined, Target returns the time that has passed since the last purchase time, divided by the number of milliseconds in a day (which results in the number of days since the last purchase).

Name: *user.frequency*

```

var frequency = user.get('frequency') || 0;
if (mbox.name == 'orderThankyouPage') {
  return frequency + 1;
}

```

Creates a variable called frequency, initializing it to either the previous value or 0, if there was no previous value. If the mbox name is orderThankyouPage, the incremented value is returned.

Name: *user.monetaryValue*

```

var monetaryValue = user.get('monetaryValue') || 0;
if (mbox.name == 'orderThankyouPage') {
  return monetaryValue + parseInt(mbox.param('orderTotal'));
}

```

Creates a variable called monetaryValue, looking up the current value for a given visitor (or set to 0 if there was no previous value). If the mbox name is orderThankyouPage, new monetary value is returned by adding the previous one and the value of the orderTotal parameter passed to the mbox.

Name: *adobeQA*

```

if (page.param("adobeQA"))
  return page.param("adobeQA");
else if (page.param("adobeqa"))
  return page.param("adobeqa");
else if (mbox.param("adobeQA"))
  return mbox.param("adobeQA");

```

Cre: **Was this content helpful?**

Detailed feedback
options



Objects and methods

The following objects and methods can be referenced by script profile parameters:

Object or Method	Details
<code>page.url</code>	The current URL.
<code>page.protocol</code>	The protocol used for the page (http or https).
<code>page.domain</code>	The current URL domain (everything before the first slash). For example, <code>www.acme.com</code> in <code>http://www.acme.com/categories/men_jeans?color=blue&size=small</code> .
<code>page.query</code>	The query string for the current page. Everything after the '?'. For example, <code>blue&size=small</code> in <code>http://www.acme.com/categories/mens_jeans?color=blue&size=small</code> .
<code>page.param('<par_name>')</code>	The value of the parameter indicated by <code><par_name></code> . If your current URL is Google's search page and you had inputted <code>page.param('hl')</code> , you would get "en" for the URL <code>http://www.google.com/search?hl=en&q=what+is+asdf&btnG=Google+Search</code> .
<code>page.referrer</code>	The same set of operations as above apply for referrer and landing (i.e. <code>referrer.url</code> is the url address of the referrer).
<code>landing.url</code> , <code>landing.protocol</code> , <code>landing.query</code> , and <code>landing.param</code>	Similar to that of page, but for the landing page. For the landing page URL to work as expected, set the context > browser > host.
<code>mbox.name</code>	The active mbox's name.
<code>mbox.param('<par_name>')</code>	An mbox parameter by the given name in the active mbox.

Was this content helpful?

Detailed feedback
options



Object or Method	Details
<code>profile.get('<par_name>')</code>	The client-created user profile parameter by the name <par_name>. For example, if the user sets a profile parameter named "gender", the value can be extracted using "profile.gender". Returns the value of the "profile.<par_name>" set for the current visitor; returns null if no value has been set. Note that <code>profile.get(<par_name>)</code> is qualified as a function call.
<code>user.get('<par_name>')</code>	Returns the value of the "user.<par_name>" set for the current visitor; returns null if no value has been set.
<code>user.categoryAffinity</code>	Returns the name of the best category.
<code>user.categoryAffinities</code>	Returns an array with the best categories.
<code>user.isFirstSession</code>	Returns true if it's the visitor's first session.
<code>user.browser</code>	Returns the user-agent in the HTTP header. As an example, you can create an expression target to target Safari users only: <code>if (user.browser != null && user.browser.indexOf('Safari') != -1) { return true; }</code>

Common operators

All standard JavaScript operators are present and usable. JavaScript operators can be used on strings and numbers (and other data types). A quick briefing:

Operator	Description
<code>==</code>	Indicates equality. Holds true when operands on either side are equal.
<code>!=</code>	Indicates inequality. Holds true when operands on either side are not equal

Was this content helpful?

Detailed feedback
options



Operator	Description
<	Indicates that the variable on the left is less than the variable on the right. Evaluates to false if the variables are equal.
>	Indicates that the variable on the left is greater than the variable on the right. Evaluates to false if the variables are equal.
<=	Same as < except if the variables are equal then it evaluates to true.
>=	Same as > except if the variables are equal then it evaluates to true.
&&	Logically “ANDs” the expressions to the left and right of it – is only true when both sides are true (false otherwise).
`	
//	Checks if source contains all elements from target boolean contains (Array source, Array target). // extracts substring from target (corresponding to regexp) and decodes it Array/*String*/ decode(String encoding, String regexp, String target). The feature also supports the use of constant string values, grouping (`condition1

Training video: Profile Scripts Tutorial

This video includes information about using and creating profile scripts.

- Explain what a profile script is
- Explain how a profile script is different from a profile parameter
- Create a simple profile script
- Use the Available Tokens menu to access available options

Was this content helpful?

Detailed feedback options



More help on this topic

- [What Are Visitor Profiles?](#)
- [Which Profiles, Variables, and Parameters Are Used in Target?](#)
- [How to Create Profile Scripts](#)

[Previous page](#)[Next page](#)

Was this content helpful?

Detailed feedback
options

